Reading Order Reconstruction for Mobile Document Readers

Alice Example[1] and Bob Sample
Example University

Abstract

Mobile readers struggle with documents that were typeset for paper. Two-column layouts, running headers, and hyphenated line breaks make the text hard to follow on a narrow screen. We present a reconstruction technique that recovers the reading order of scholarly documents from the positions of their glyphs and reflows the text so that it can be read comfortably on a telephone.

1 Introduction

Scholarly publishing still revolves around the portable document format. Conference proceedings, technical reports, and standards documents are distributed as fixed layouts that faithfully reproduce the printed page. This fidelity comes at a price: on a small display the reader must zoom, pan, and scroll in two dimensions to follow a single sentence across a column boundary, which interrupts concentration and makes careful reading of long specifications exhausting.

Our approach treats the document as a collection of positioned words rather than as an image. By grouping words into lines, lines into columns, and columns into a reading sequence, the system recovers the logical flow of the text. Figures, tables, and displayed equations are preserved as cropped images of the original page, because their meaning depends on the arrangement that the author chose.

The contributions of this paper are:

- a reading order algorithm that handles two-column pages and full-width titles;
- a paragraph segmentation rule based on indentation and spacing.

2 Design

The first stage collects every text fragment together with its baseline, font size, and horizontal extent. Fragments that share a baseline are merged into lines, and superscripts such as footnote markers are attached to the line they decorate. A vertical gap that remains empty across most of the page reveals the boundary between the two columns, even when a wide title or a figure spanning both columns crosses it.

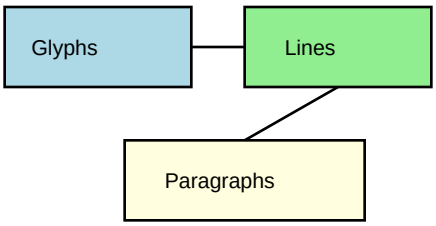

Figure 1: The three stages of the reconstruction pipeline.

The second stage decides where paragraphs begin. Typeset documents signal a new paragraph either with additional vertical space or with an indented first line. Both cues are measured relative to the typical line spacing of the body text, so the method adapts to documents with different typographic conventions without manual configuration.

The score of a candidate column boundary is computed as follows:

$$S = \sum_{i} w_i \cdot d(x_i, y_i) + \lambda \tag{1}$$

2.1 Running Headers and Hyphenation

Running headers and page numbers repeat on every page and carry no information for the reader. They are recognized by comparing the text near the top and bottom margins across pages, ignoring the digits that change from one page to the next. Hyphenated words at the end of a line are rejoined unless the same hyphenated form appears elsewhere in the document, which preserves compound words such as state-of-the-art techniques while repairing words that were merely broken for justification.

The detection itself is a short function:

```
function isHeader(line, pages) {
  const key = normalize(line.text);
  return count(key, pages) >= 3;
```

1 Work done while the author was visiting the Example Laboratory for Document Engineering, which provided the test collection used in this study.

```
}
```

Listing 1: Detecting running headers.

3 Evaluation

We evaluated the reconstruction on a collection of conference papers and technical reports. For each document we compared the recovered paragraphs with the source text and counted the paragraphs whose words appeared in the correct order. The comparison was strict: a single misplaced fragment caused the entire paragraph to be counted as incorrect.

Table 1: Paragraphs whose reading order was recovered.

Corpus	Docs	Correct
Conference papers	120	98.3%
Technical reports	45	96.1%
Standards	30	99.0%

The results in Table 1 show that the reading order was recovered for nearly every paragraph. The remaining errors were concentrated in documents that place marginal notes beside the main text, a layout that the current implementation does not attempt to handle.

4 Conclusion

Reflowing fixed layouts is an old problem, but the combination of glyph positions, font metrics, and simple statistics about the page is sufficient for the documents that researchers read most often. The resulting text adapts to the reader instead of forcing the reader to adapt to the page.

References

[1] T. M. Breuel. Two geometric algorithms for layout analysis. In Document Analysis Systems, pages 188–199. Springer, 2002.

[2] R. Smith. Hybrid page layout analysis via tab-stop detection. In International Conference on Document Analysis and Recognition, pages 241–245, 2009. https://research.example.org/papers/hybrid-page-layout-analysis-tab-stops.pdf

[3] Mozilla. PDF.js: a general-purpose, web standards-based platform for parsing and rendering PDFs, 2024.